trvxluxe

TRVXLUXE.DE

THE COMPLETE

Programming Course

A practical e-book and workbook. From your first line of code to real projects, in four levels.

```python
def launch(project):
    return ship(project)

# level 1 → real projects
print("Hello, TRVXLUXE!")
```

01	02	03	04
FOUNDATIONS	BUILDING BLOCKS	REAL DEVELOPER	REAL PROJECTS

START HERE

Welcome to Programming

WHAT PROGRAMMING MEANS

Programming means turning a problem into clear instructions a computer can execute. Code can automate repetitive work, process data, power websites and build applications. The goal is not to memorize every command. Learn to break problems into small steps, test assumptions and explain your solution.

HOW TO USE THIS COURSE

Type the examples yourself. Predict what they will do before running them, then change values and test again. Keep a folder for exercises and projects. When an error appears, read it carefully and change one thing at a time.

LEARNING PATH

This course begins with Python, then covers data structures, functions, files, debugging, testing, Git, HTML, CSS, JavaScript, APIs, databases, security and practical projects. You can later specialize in web development, automation or data work.

YOUR PATH



LEVEL 1
Foundations
Set up Python and write your first working programs.

LEVEL 2
Building Blocks
Organise data and code so that your programs stay readable.

LEVEL 3
Real Developer
Work the way developers work: classes, tests, Git, the web and databases.

LEVEL 4
Real Projects
Plan, secure and finish projects you can show to other people.

Please note. This is educational material. It does not guarantee employment, income or professional certification. Tools and platforms change, so check the current official documentation.

© 2026 trvxluxe.de · All rights reserved. This copy is for your personal use. Please do not share or resell it.

Contents

LEVEL 1 Foundations

01 Setup and Your First Program 05
02 Variables and Data Types 06
03 Input, Output and Operators 07
04 Conditions and Decision Making 08
05 Loops 09

LEVEL 2 Building Blocks

06 Lists, Tuples, Sets and Dictionaries 11
07 Functions and Reusable Code 12
08 Errors, Debugging and Exceptions 13
09 Files, JSON and Data Persistence 14
10 Modules, Packages and Environments 15

LEVEL 3 Real Developer

11 Object-Oriented Programming 17
12 Testing and Code Quality 18
13 Git and GitHub 19
14 HTML, CSS and JavaScript 20
15 APIs, HTTP and JSON 22
16 Databases and SQL 23

LEVEL 4 Real Projects

17 Building a Web Application 25
18 Security, Privacy and Responsible Development 26
19 Automation with Python 27
20 Intermediate Python 28
21 Project One: Command-Line Task Manager 29
22 Project Two: Portfolio Website 30
23 Project Three: Product Price Tracker 31

EXTRAS Keep Growing

24 Debugging and Learning Effectively 33
25 Roadmap from Beginner to Real Projects 34
26 Glossary 35
27 Final Assessment and Workbook 36

trvxluxe

01

LEVEL 1 OF 4

Foundations

Set up Python and write your first working programs.

BY THE END YOU SHOULD BE ABLE TO

- Install Python, run a program and read what it prints.
- Use variables, input, conditions and loops to solve small problems.
- Read an error message and fix it one change at a time.

CHAPTERS

01 Setup and Your First Program

02 Variables and Data Types

03 Input, Output and Operators

04 Conditions and Decision Making

05 Loops

01

LEVEL 1 · FOUNDATIONS

Setup and Your First Program

TOOLS

Use a current Python 3 release from the official Python website or a trusted learning environment. A code editor such as Visual Studio Code can help, but you can start in an online editor. Download tools only from trusted sources.

FIRST PROGRAM

PYTHON

```python
print("Hello, TRVXLUXE!")
```

HOW IT WORKS

`print()` is a built-in function that displays information. Text inside quotation marks is a string. Parentheses contain the information passed to the function. Python runs statements in order.

PRACTICE

Print your name, one goal and one sentence about what you want to build. Put each message on a separate line. Then deliberately remove a quotation mark and inspect the error.

02

LEVEL 1 · FOUNDATIONS

Variables and Data Types

VARIABLES

A variable is a name that refers to a value. Example: username = "Alex". The equals sign assigns a value; it does not mean mathematical equality here.

CORE TYPES

`str` stores text, `int` stores whole numbers, `float` stores decimal numbers and `bool` stores `True` or `False`. Use `type(value)` to inspect a value.

EXAMPLE

PYTHON

```python
username = "Alex"
age = 16
height_m = 1.88
is_learning = True
print(username)
print(age)
print(type(height_m))
```

NAMING

Use clear names such as `total_price` or `user_age`. Python names are case-sensitive. Lowercase words separated by underscores are common style.

PRACTICE

Create variables for a product name, price, quantity and availability. Print a summary using all four.

03

LEVEL 1 · FOUNDATIONS

Input, Output and Operators

USER INPUT

`input()` returns what the user types as text. Convert it to `int` or `float` before doing numeric calculations.

EXAMPLE

PYTHON

```python
name = input("Your name: ")
number = float(input("Choose a number: "))
print(f"Hello, {name}. Double is {number * 2}.")
```

OPERATORS

Arithmetic: +, -, *, /, //, %, **. Comparisons: ==, !=, >, <, >=, <=. Logic: and, or, not. Parentheses can make complex expressions clearer.

F-STRINGS

An f-string starts with f before its opening quote. Expressions inside curly braces are evaluated and inserted into the output.

> **PRACTICE**
>
> Build a calculator that asks for two numbers and prints their sum, difference, product and quotient. Think about what should happen if the second number is zero.

04

LEVEL 1 · FOUNDATIONS

Conditions and Decision Making

IF, ELIF, ELSE

Conditions let a program choose between paths. Python uses indentation to define each block, normally four spaces.

EXAMPLE

PYTHON

```python
score = int(input("Score from 0 to 100: "))
if score >= 90:
    print("Excellent")
elif score >= 70:
    print("Good")
elif score >= 50:
    print("Pass")
else:
    print("Keep practising")
```

VALIDATION

This example assumes valid integer input. A robust program should reject values outside the expected range and handle invalid input.

> **PRACTICE**
>
> Create a program that says whether a number is positive, negative or zero, then checks whether it is even or odd.

YOUR NOTES

05

LEVEL 1 · FOUNDATIONS

Loops

FOR LOOPS

A for loop repeats code for each item in a sequence. `range(5)` produces 0, 1, 2, 3 and 4.

WHILE LOOPS

A while loop repeats while a condition is true. Make sure the condition can eventually become false, or the program may run forever.

EXAMPLE

PYTHON

```python
for number in range(1, 6):
    print(number)
count = 3
while count > 0:
    print(count)
    count -= 1
print("Go!")
```

CONTROL FLOW

break exits a loop; continue skips to the next iteration. Use these when they make the logic easier to understand.

> **PRACTICE**
>
> Print a multiplication table for a user-selected number. Then build a guessing game with a limited number of attempts.

trvxluxe

02

LEVEL 2 OF 4

Building Blocks

Organise data and code so that your programs stay readable.

BY THE END YOU SHOULD BE ABLE TO

- Store and change data with lists, sets and dictionaries.
- Write reusable functions and handle errors without crashing.
- Save data to files and JSON, and keep projects tidy with virtual environments.

CHAPTERS

06 Lists, Tuples, Sets and Dictionaries

07 Functions and Reusable Code

08 Errors, Debugging and Exceptions

09 Files, JSON and Data Persistence

10 Modules, Packages and Environments

06

LEVEL 2 · BUILDING BLOCKS

Lists, Tuples, Sets and Dictionaries

LISTS

Lists are ordered and changeable. Example: fruits = ["apple", "banana"]. Indexes start at zero, so fruits[0] is the first item.

TUPLES AND SETS

Tuples are ordered collections often used for values that should not change. Sets store unique values and are useful for removing duplicates or checking membership.

DICTIONARIES

Dictionaries map keys to values. Example: product = {"name": "Notebook", "price": 4.99}. Access a value with product["name"].

EXAMPLE

PYTHON

```python
products = [{"name": "Notebook", "price": 4.99}, {"name": "Pen", "price": 1.50}]
for product in products:
    print(f"{product['name']}: €{product['price']:.2f}")
```

PRACTICE

Create a five-item inventory. Add an item, update a price, remove an item and print the remaining products.

07

LEVEL 2 · BUILDING BLOCKS

Functions and Reusable Code

WHY FUNCTIONS MATTER

Functions group instructions under a name so they can be reused and tested. They reduce repetition and make programs easier to read.

EXAMPLE

PYTHON

```python
def calculate_total(price, quantity):
    return price * quantity
subtotal = calculate_total(12.50, 3)
print(subtotal)
```

PARAMETERS AND RETURNS

Parameters are inputs defined by a function. return sends a result back to the caller. print displays something but does not replace return.

SCOPE AND STYLE

Variables created inside a function are generally local to it. Use descriptive names and short docstrings where useful. Pass data explicitly rather than relying on hidden global state.

> **PRACTICE**
>
> Write functions for adding an item, calculating a cart total and applying a discount. Test each with different values.

08

LEVEL 2 · BUILDING BLOCKS

Errors, Debugging and Exceptions

ERROR TYPES

Syntax errors stop Python parsing the code. Runtime errors occur during execution. Logic errors mean the program runs but produces the wrong result.

TRACEBACKS

Read the final line for the exception type and message, then inspect the referenced line and nearby code. The real cause may occur earlier.

EXAMPLE

PYTHON

```python
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a whole number.")
else:
    print(f"You entered {age}.")
```

DEBUGGING WORKFLOW

Reproduce the problem, read the error, identify assumptions, isolate a small example, make one change and test again. Avoid rewriting everything randomly.

> **PRACTICE**
>
> Update your calculator to handle invalid numbers and division by zero without crashing.

09

LEVEL 2 · BUILDING BLOCKS

Files, JSON and Data Persistence

WORKING WITH FILES

Use with `open(...)` so files are closed automatically. The mode r reads, w writes and replaces existing content, and a appends. Write mode can erase old content, so use it carefully.

EXAMPLE

PYTHON

```python
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("My first saved note\n")
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
print(content)
```

JSON

JSON is a common text format for structured data. Python's `json` module converts supported Python data to and from JSON. Avoid loading untrusted serialized formats such as `pickle`.

> **PRACTICE**
>
> Build a task list that saves tasks to a JSON file and reloads them the next time the program starts.

YOUR NOTES

10

LEVEL 2 · BUILDING BLOCKS

Modules, Packages and Environments

MODULES

A module is a Python file or library that provides reusable functionality. Example: import random lets you use random-number functions.

PACKAGES AND PIP

Packages group modules. `pip` installs third-party packages. Check package names and publishers before installing them.

VIRTUAL ENVIRONMENTS

A virtual environment isolates project dependencies so different projects do not interfere with each other. Create one for each substantial project and document the setup.

SECURITY

Do not install packages from random links or run code you do not understand. Dependencies can contain vulnerabilities or malicious code.

> **PRACTICE**
>
> Create a project folder, a virtual environment and a short README explaining how to reproduce the setup.

trvxluxe

03

LEVEL 3 OF 4

Real Developer

Work the way developers work: classes, tests, Git, the web and databases.

BY THE END YOU SHOULD BE ABLE TO

- Structure code with classes and check it with simple tests.
- Track your work with Git and build a basic web page with HTML, CSS and JavaScript.
- Call an API and store data with SQL.

CHAPTERS

11 Object-Oriented Programming

12 Testing and Code Quality

13 Git and GitHub

14 HTML, CSS and JavaScript

15 APIs, HTTP and JSON

16 Databases and SQL

11

LEVEL 3 · REAL DEVELOPER

Object-Oriented Programming

CLASSES AND OBJECTS

A class defines a type of object; an object is an instance of that class. Classes can group related data and behavior.

EXAMPLE

PYTHON

```python
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def describe(self):
        return f"{self.name} costs €{self.price:.2f}"
item = Product("Notebook", 4.99)
print(item.describe())
```

SELF AND __INIT__

`self` refers to the current instance. `__init__` initializes its attributes. Use classes when they make the design clearer, not just because they exist.

> **PRACTICE**
>
> Create Product and ShoppingCart classes. Add products to a cart and calculate the total.

12

LEVEL 3 · REAL DEVELOPER

Testing and Code Quality

WHY TEST

Tests verify expected behavior and help prevent old bugs returning. Manual checks are useful; automated tests are repeatable.

EXAMPLE

PYTHON

```python
def add(a, b):
    return a + b
assert add(2, 3) == 5
assert add(-1, 1) == 0
```

READABLE CODE

Use meaningful names, consistent indentation, small functions and comments that explain why a decision exists. Avoid comments that merely repeat the code.

REFACTORING

Refactoring changes internal structure without intending to change behavior. Run tests before and after refactoring.

> **PRACTICE**
>
> Write at least five tests for your calculator, including normal inputs, negative values, zero and relevant invalid cases.

13

LEVEL 3 · REAL DEVELOPER

Git and GitHub

VERSION CONTROL

Git records changes to files. It helps compare versions, recover earlier work and collaborate. GitHub hosts Git repositories.

COMMON WORKFLOW

TERMINAL

```
git init
git status
git add .
git commit -m "Add first project version"
git log --oneline
```

COMMITS

Make small commits with clear messages. A README should explain what the project does, how to install it, how to run it and how to test it.

SECRETS

Never commit passwords, API keys or private customer data. Use .gitignore for local secret files and rotate any secret that becomes exposed.

> **PRACTICE**
>
> Create a repository for your task-list project, make several commits and write a README.

14

LEVEL 3 · REAL DEVELOPER

HTML, CSS and JavaScript

THREE ROLES

HTML structures page content, CSS controls presentation and JavaScript adds behavior in the browser.

MINIMAL HTML

HTML

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, world!</h1>
  <p>This is my first web page.</p>
  <script src="script.js"></script>
</body>
</html>
```

CSS EXAMPLE

CSS

```css
body {
  font-family: system-ui, sans-serif;
  max-width: 720px;
  margin: 3rem auto;
  padding: 0 1rem;
}
```

JAVASCRIPT EXAMPLE

JAVASCRIPT

```javascript
const button = document.querySelector("#hello-button");
if (button) {
  button.addEventListener("click", () => {
    alert("Hello!");
  });
}
```

RESPONSIVE AND ACCESSIBLE

Use flexible layouts and media queries so pages work on phones and desktops. Check keyboard navigation, contrast, labels and readable text.

> **PRACTICE**
>
> Build a responsive portfolio with a navigation bar, about section, three projects and a contact link.

YOUR NOTES

15

LEVEL 3 · REAL DEVELOPER

APIs, HTTP and JSON

WHAT AN API IS

An API defines how software systems exchange information. A web API often receives HTTP `requests` and returns JSON.

HTTP BASICS

GET commonly retrieves data; POST commonly submits it; PUT or PATCH may update it; DELETE may remove it. Follow the API's documented contract.

EXAMPLE

PYTHON

```python
import requests
response = requests.get("https://api.github.com", timeout=10)
response.raise_for_status()
data = response.json()
print(data.get("current_user_url", "No field found"))
```

SAFE USE

Read API documentation, respect rate limits, set timeouts and handle errors. Never expose private API keys in frontend code or public repositories.

> **PRACTICE**
>
> Use a documented public API to retrieve a small dataset. Handle timeouts, unsuccessful HTTP responses and missing fields.

16

LEVEL 3 · REAL DEVELOPER

Databases and SQL

WHY DATABASES

Databases store and organize information so it can be queried and updated. SQLite is useful for learning and small local apps.

SQL BASICS

SELECT reads rows, INSERT adds rows, UPDATE changes rows and DELETE removes rows. Be careful with UPDATE or DELETE statements without a WHERE clause.

EXAMPLE

PYTHON

```python
import sqlite3
connection = sqlite3.connect("shop.db")
try:
    connection.execute("CREATE TABLE IF NOT EXISTS products (id INTEGER PRIMARY KEY, name TEXT, price REAL)")
    connection.execute("INSERT INTO products (name, price) VALUES (?, ?)", ("Notebook", 4.99))
    connection.commit()
    rows = connection.execute("SELECT name, price FROM products").fetchall()
    print(rows)
finally:
    connection.close()
```

PARAMETERIZED QUERIES

Use placeholders and parameter values rather than concatenating user input into SQL. This helps prevent SQL injection.

> **PRACTICE**
>
> Create a database for tasks or products. Add, list, update and delete records. Test what happens when a record does not exist.

trvxluxe

04

LEVEL 4 OF 4

Real Projects

Plan, secure and finish projects you can show to other people.

BY THE END YOU SHOULD BE ABLE TO

- Plan a small web application and apply security basics.
- Automate repetitive tasks safely with Python.
- Finish three complete projects: a task manager, a portfolio site and a price tracker.

CHAPTERS

17 Building a Web Application

18 Security, Privacy and Responsible Development

19 Automation with Python

20 Intermediate Python

21 Project One: Command-Line Task Manager

22 Project Two: Portfolio Website

23 Project Three: Product Price Tracker

17

LEVEL 4 · REAL PROJECTS

Building a Web Application

FRONTEND AND BACKEND

The frontend is the part users interact with. A backend handles server-side logic, authentication and database access. Start with a static website before adding a backend.

PLAN FIRST

Define the user problem, minimum useful features, data needs, screens and risks. Build a small working version before optional features.

EXAMPLE STRUCTURE

PROJECT STRUCTURE

```
my_app/
  README.md
  app.py
  requirements.txt
  templates/
    index.html
  static/
    style.css
```

SECURITY ESSENTIALS

Validate data on the server, store passwords with a reputable password-hashing library, protect secrets in environment variables, use HTTPS and check authorization for private resources. Never invent your own cryptography.

> **PRACTICE**
>
> Build a notes or task application. Before using real users or sensitive data, identify what security, privacy and reliability improvements are needed.

18

LEVEL 4 · REAL PROJECTS

Security, Privacy and Responsible Development

VALIDATE INPUTS

Treat data from users, files, URLs and external services as untrusted. Check its type, length, format and allowed values.

PROTECT SECRETS

Never hard-code passwords or secret API keys in code that may be shared. Limit permissions and rotate exposed credentials.

PRIVACY

Collect only the data needed, explain how it is used, restrict access and define how it can be deleted. Requirements vary by region and data type.

ETHICAL BOUNDARIES

Only test systems you own or have explicit permission to assess. Do not build malware, steal credentials or access another person's data without authorization.

> **PRACTICE**
>
> Review one project. List its inputs, stored data, secrets, dependencies and failure modes, then prioritize improvements.

19

LEVEL 4 · REAL PROJECTS

Automation with Python

GOOD AUTOMATION TARGETS

Automation works best for repetitive, clearly defined tasks such as listing files, generating reports or organizing personal data.

EXAMPLE

PYTHON

```python
from pathlib import Path
folder = Path("sample_files")
for path in folder.glob("*.txt"):
    print(path.name, path.stat().st_size, "bytes")
```

SAFETY

Test scripts on copies of sample data. For scripts that rename, delete, send or publish information, use dry-run mode, confirmations, logs and backups.

> **PRACTICE**
>
> Create a report listing file names and sizes in a test folder and save it to a new file. Do not overwrite the originals.

20

LEVEL 4 · REAL PROJECTS

Intermediate Python

COMPREHENSIONS

A list comprehension creates a list from an iterable. Example: squares = [n * n for n in range(1, 6)]. Use it when it is clearer than a normal loop.

SORTING AND FILTERING

sorted(items, key=...) sorts using a chosen field. Keep filtering rules easy to understand and test.

TYPE HINTS

PYTHON

```python
def calculate_average(values: list[float]) -> float:
    if not values:
        raise ValueError("values must not be empty")
    return sum(values) / len(values)
print(calculate_average([4.0, 6.0, 8.0]))
```

ITERATORS AND GENERATORS

An iterator produces values one at a time. A generator uses yield to produce values lazily and can help when processing large sequences.

> **PRACTICE**
>
> Filter product prices above a threshold, sort them, calculate an average, add type hints and write tests.

YOUR NOTES

21

LEVEL 4 · REAL PROJECTS

Project One: Command-Line Task Manager

REQUIREMENTS

Let a user add a task, list tasks, mark a task complete and save tasks between runs. Start with a simple text menu.

DESIGN

Represent each task with an ID, title and completed status. Separate user input from the functions that manipulate task data. Save to JSON and handle missing or malformed files.

BUILD SEQUENCE

First add and list tasks in memory. Next add completion status, then persistence, validation, tests and a README.

ACCEPTANCE CHECKLIST

- [ ] Add a task.
- [ ] List tasks.
- [ ] Mark a task complete.
- [ ] Data remains after restarting.
- [ ] Invalid IDs are handled.
- [ ] Malformed data does not silently destroy the original.
- [ ] Tests cover core functions.

EXTENSIONS

Add due dates, categories, search, sorting, CSV export or a web interface. Add one feature at a time.

22

LEVEL 4 · REAL PROJECTS

Project Two: Portfolio Website

GOAL

Build a clean site presenting your skills and projects. Avoid publishing private information such as your home address or personal phone number.

SECTIONS

Include a hero section, introduction, three projects, skills and a contact link. Keep navigation simple and layout responsive.

WORKFLOW

Sketch the layout, write semantic HTML, add CSS, add only necessary JavaScript, test phone and desktop widths, check accessibility and deploy with a trusted host.

PROJECT CARDS

Explain the problem, what you built, technologies used and what you learned. Be honest about your contribution and avoid invented testimonials.

ACCEPTANCE CHECKLIST

- [ ] Links work.
- [ ] Mobile layout works.
- [ ] Text is readable.
- [ ] Images have suitable alternative text.
- [ ] No secrets are exposed.
- [ ] Title and metadata are set.
- [ ] README explains updates.

23

LEVEL 4 · REAL PROJECTS

Project Three: Product Price Tracker

GOAL

Create a prototype that stores manually entered product prices and shows changes over time. Begin with your own data or a permitted public API.

DATA MODEL

Store product name, price, currency, date and source. Validate prices and dates; do not mix currencies without a clear conversion method.

FEATURES

Add a price record, list history, calculate change from the previous record and export CSV. Add charts only after the underlying data is correct.

RESPONSIBLE COLLECTION

Before automating website collection, read the site's terms, respect rate limits and privacy, and prefer official APIs. Do not bypass access controls.

ACCEPTANCE CHECKLIST

- [ ] Invalid prices are rejected.
- [ ] History persists.
- [ ] Currency is visible.
- [ ] Calculations are tested.
- [ ] Errors are clear.
- [ ] Sources and limitations are documented.

trvxluxe



EXTRAS

Keep Growing

Debug like a pro, follow the roadmap and test yourself.

CHAPTERS

24 Debugging and Learning Effectively

25 Roadmap from Beginner to Real Projects

26 Glossary

27 Final Assessment and Workbook

24

KEEP GROWING

Debugging and Learning Effectively

THE DEBUGGING LOOP

Reproduce the problem. Read the full error. Identify the smallest relevant area. State a hypothesis. Make one change. Run the test again. Record the fix.

SEARCH EFFECTIVELY

Search using the exact error message and relevant library version. Prefer official documentation and reputable sources. Understand a proposed fix before using it.

AI-ASSISTED CODING

AI can explain errors and suggest tests, but it can also invent APIs or produce insecure code. Ask for explanations, verify claims, run tests and inspect every important change.

AVOID TUTORIAL DEPENDENCE

After a tutorial, close it and rebuild the core feature from memory. Change the requirements slightly. If you cannot explain the code, return to that concept.

> **PRACTICE**
>
> Try a short repeatable session: review a concept, write code, debug it, then record what you learned. Consistency matters more than occasional marathon sessions.

25

KEEP GROWING

Roadmap from Beginner to Real Projects

STAGE 1: FOUNDATIONS

Learn variables, conditions, loops, collections, functions, files and exceptions. Build small command-line programs without copying complete solutions.

STAGE 2: CORE DEVELOPMENT

Learn Git, testing, debugging, virtual environments, dependency management and basic data structures. Finish two projects with documentation.

STAGE 3: CHOOSE A PATH

Web: deepen HTML, CSS, JavaScript, HTTP, a backend framework and databases. Automation: focus on Python, files, APIs and scheduling. Data: learn cleaning, SQL, visualization and statistics.

STAGE 4: PORTFOLIO

Publish a few complete projects with clear READMEs, tests, setup instructions and honest limitations. Quality and understanding matter more than many unfinished repositories.

REALITY CHECK

Completing a course does not automatically make someone job-ready. Employers and clients value demonstrable skills, reliability, communication, problem solving and maintainable software.

26

KEEP GROWING

Glossary

Algorithm
A defined sequence of steps for solving a problem.

API
An interface that allows software systems to communicate.

Argument
A value passed to a function when it is called.

Boolean
A value that is either `True` or `False`.

Bug
A defect that causes software to behave incorrectly.

Class and object
A class defines a type; an object is an instance of that type.

Database
An organized store of data that can be queried and updated.

Exception
An error or unusual condition during program execution.

Function
A named reusable block of code.

Git
A version-control system for tracking changes.

HTTP
A protocol used for communication on the web.

JSON
A text format commonly used to exchange structured data.

Library
Reusable code that a program can import.

Loop
A construct that repeats a block of code.

Module and package
A module is a code unit; packages organize modules or distribute them.

Parameter
A named input defined by a function.

SQL
A language used to work with relational databases.

Test
A check that verifies expected behavior.

Variable
A name referring to a value.

Version control
A way to record and manage file changes over time.

27

KEEP GROWING

Final Assessment and Workbook

KNOWLEDGE CHECK

1. What is the difference between a variable and a function?
2. Why does `input()` usually need conversion before arithmetic?
3. Explain == versus =.
4. When would you use a dictionary instead of a list?
5. What does a traceback help you find?
6. Why use a virtual environment?
7. What is the risk of publishing an API key?
8. Why use parameterized SQL queries?
9. What is a unit test?
10. Why verify AI-generated code?

PROJECT SUBMISSION TEMPLATE

Project name	Problem solved
Intended user	Main features
Technologies	Setup steps
How to run	How to test
Known limitations	Security and privacy considerations
What I learned	Next improvement

WEEKLY LEARNING JOURNAL

Week

Concepts practised

Code written without copying

Errors solved

What is unclear

Project progress

Next specific goal

COMPLETION STANDARD

You are progressing when you can explain your code, modify it without blindly following a tutorial, test important behavior, diagnose common errors and document how another person can run it.

PLEASE READ

Responsible Use and Publisher Disclaimer

EDUCATIONAL SCOPE

This course is a general educational resource. It does not guarantee employment, income, business success or mastery of programming. Progress depends on practice, prior knowledge and project complexity.

SECURITY AND LEGALITY

Follow applicable laws, website terms, software licences and privacy requirements. Only access systems and data you are authorized to use. Do not deploy code you do not understand where it could cause harm.

SOFTWARE CHANGES

Languages, libraries, APIs, hosting platforms and installation steps change. Check current official documentation before using version-specific commands in production.

trvxluxe

THANK YOU

Keep going.
One step at a time.

Finish a level, do the exercises and review your work before you move on. Consistency beats speed.

trvxluxe.de · trvluxe56@gmail.com